

P1018R6

Language Evolution status after Prague 2020

Published Proposal, 2020-02-29

This version:

<http://wg21.link/P1018r6>

Author:

[JF Bastien](#) (Apple)

Audience:

WG21, EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/jfbastien/papers/blob/master/source/P1018r6.bs

Abstract

This paper is a collection of items that language Evolution has worked on in the latest C++ meeting, their status, and plans for the future.

Table of Contents

- 1 Executive summary
- 2 Paper of note
- 3 Tentatively ready papers
- 4 ABI discussion

5 National body comments

6 C++23 discussions

7 Near-future EWG plans

References

Informative References

§ 1. Executive summary

- Finalize ballot resolution for C++20, to address National Body comments in [\[N4844\]](#).
- Start work on features for C++23 and later.
- Joins session with LEWG on ABI, based on [P1863R1](#).

§ 2. Paper of note

- [P1000R4](#) C++ IS schedule
- [P0592R4](#) To boldly suggest an overall plan for C++23
- [P1999R0](#) Process: 2×- evolutionary material via a Tentatively Ready status
- [P2118R0](#) Documenting Core Undefined or Unspecified Behavior

§ 3. Tentatively ready papers

Following our process in [P1999](#), here are the papers that EWG considers tentatively ready for CWG. We'll take a brief look at the next meeting, and if nothing particular concerns anyone, send them to CWG.

- [P1847R2](#) Make declaration order layout mandated
- [P2025R0](#) Guaranteed copy elision for named return objects
- [P1949R2](#) C++ Identifier Syntax using Unicode Standard Annex 31

You can [follow this list on GitHub](#).

§ 4. ABI discussion

We held a joint sessions with LEWG to discuss ABI, based on [P1863R1](#), The outcome of the discussion was as follows:

- **To the best of our ability, we should promise users that we won't break ABI, ever**
Wasn't contended: we disagree with this statement and might break ABI in the future.
- **From now on, we should consider incremental ABI for every C++ release**
Received extremely positive support, with a small minority disagreeing strongly.
- **We should consider a big ABI break for C++23**
Was extremely contended, with a few more people in favor than against. This was insufficient to call consensus.
- **We should consider a big ABI break for C++SOMETHING**
Was positive enough to call consensus, but still had a quite substantial opposition including many disagreeing strongly. Were we to do a big ABI break we would need to work very hard on consensus building. Indeed, the number of people disagreeing strongly on a poll for a concrete change would block consensus.
- **When we are unable to resolve a conflict between performance and ABI compatibility, we should prioritize performance**
Was still more positive, but also had a quite substantial opposition including many disagreeing strongly. Again, we should consider performance over ABI but work extremely hard towards consensus building when doing so.

§ 5. National body comments

- [P2003R0](#) Fixing Internal and External Linkage Entities in Header Units [#740](#)
- [P2014R0](#) Proposed resolution for US061/US062 - aligned allocation of coroutine frames [#750](#)
- [P1884R0](#) Private Module Partition: An Inconsistent Boundary [#729](#)
- [P2100R0](#) Keep unhandled_exception of a promise type mandatory - a response to US062 and FR066
- [P2104R0](#) GB046 Allow caching of evaluations of concept specializations [#45](#)

§ 6. C++23 discussions

We discussed a few papers which could make it to C++23:

- [P2085R0](#) Consistent defaulted comparisons
- [P0592R4](#) To boldly suggest an overall plan for C++23
- [P1999R0](#) Process proposal: double-check evolutionary material via a Tentatively Ready status
- [P1468R3](#) Fixed-layout floating-point type aliases
- [P1467R3](#) Extended floating-point types
- [P1371R2](#) Pattern Matching
- [P1000R4](#) C++ IS schedule
- [P1726R2](#) Pointer lifetime-end zap
- [P2092R0](#) Disambiguating Nested-Requirements
- [P1040R5](#) std::embed
- [P1677R2](#) Cancellation is not an Error
- [P1401R2](#) Narrowing contextual conversions to bool
- [P0876R10](#) fiber_context - fibers without scheduler
- [P0847R4](#) Deducing this
- [P2082R1](#) Fixing CTAD for aggregates
- [P1774R3](#) Portable assumptions
- [P2118R0](#) Documenting Core Undefined or Unspecified Behavior
- [P0849R2](#) auto(x): decay-copy in the language
- [P2036R0](#) Changing scope for lambda trailing-return-type
- [P2071R0](#) Named universal character escapes
- [P1900R0](#) Concepts-Adjacent Problems
- [P1847R2](#) Make declaration order layout mandated
- [P1393R0](#) A General Property Customization Mechanism
- [P2026R0](#) A Constituent Study Group for Safety-Critical Applications
- [P1938R0](#) if consteval
- [P1955R0](#) Top Level Is Constant Evaluated
- [P2041R0](#) Deleting variable templates
- [P0870R2](#) A proposal for a type trait to detect narrowing conversions
- [P2025R0](#) Guaranteed copy elision for named return objects

- [P2013R0](#) Freestanding Language: Optional ::operator new
- [P1949R2](#) C++ Identifier Syntax using Unicode Standard Annex 31

The following papers were scheduled for discussion, but authors requested to delay until the next meeting:

- [P1967R1](#) #embed - a simple, scannable preprocessor-based resource acquisition method
- [P1046R2](#) Automatically Generate More Operators
- [P2049R0](#) Constraint refinement for special-cased functions

The following papers were scheduled for discussion, but were seen in SG7 Reflection who decided to table them for now:

- [P1733R0](#) User-friendly and Evolution-friendly Reflection: A Compromise
- [P2089R0](#) Function parameter constraints are fragile

§ 7. Near-future EWG plans

We will continue to work on C++23, prioritizing according to [P0592](#).

§ References

§ Informative References

[N4844]

Barry Hedquist. [Collated CD 14882 Comments](#). 22 October 2019. URL: <https://wg21.link/n4844>